

LeetCode Python Cheat Sheet: Quant Print Version

This is the short version for quant-style interview review. The goal is fast pattern selection, clean Python templates, and the highest-yield ideas.

1-Minute Triage

If you see this	Use this first
Sorted array	Binary Search, Two Pointers
Pair / triplet / partition on ordered data	Two Pointers
Subarray / substring with local rule	Sliding Window
Exact subarray sum / count	Prefix Sum + HashMap
Top K / streaming rank	Heap
One kth element only	QuickSelect
Unweighted shortest path	BFS
Weighted shortest path, non-negative edges	Dijkstra
DAG dependency order	Topological Sort
Best value over many choices	DP
Local move seems always safe	Greedy
Fast lookup / counting / dedupe	HashMap / Set
Must implement an API	Design + right data structure

What To Prioritize

If time is short, over-practice these:

- HashMap / Set
- Sorting + Two Pointers
- Sliding Window
- Prefix Sum + HashMap
- Binary Search
- Heap
- BFS / Graph traversal
- Topological Sort
- Dynamic Programming
- Greedy
- Design Problems

Core Patterns

1. HashMap / Set

Use when

- Need lookup, counting, dedupe, or complement search.

Core idea

Store the repeated lookup instead of recomputing it.

```
def two_sum(nums, target):
    seen = {}
    for i, x in enumerate(nums):
        if target - x in seen:
            return [seen[target - x], i]
        seen[x] = i
    return []
```

- Time: $O(n)$
- Space: $O(n)$
- Memory rule: repeated lookup -> dict / set

2. Two Pointers

Use when

- Sorted array, palindrome, pair sum, compaction, partition.

Core idea

Move one boundary at a time to eliminate impossible answers.

```
def two_sum_sorted(nums, target):
    left, right = 0, len(nums) - 1
    while left < right:
        total = nums[left] + nums[right]
        if total == target:
            return [left, right]
        if total < target:
            left += 1
        else:
            right -= 1
    return [-1, -1]
```

- Time: $O(n)$
- Space: $O(1)$
- Memory rule: ordered scan -> two pointers

3. Sliding Window

Use when

- Subarray or substring with a local validity rule.

Core idea

Grow right, shrink left, maintain a valid window.

```
from collections import defaultdict

def length_of_longest_substring(s):
    freq = defaultdict(int)
    left = 0
    ans = 0

    for right, ch in enumerate(s):
        freq[ch] += 1
        while freq[ch] > 1:
            freq[s[left]] -= 1
            left += 1
        ans = max(ans, right - left + 1)

    return ans
```

- Time: $O(n)$
- Space: $O(k)$
- Memory rule: local contiguous validity -> window

4. Prefix Sum + HashMap

Use when

- Exact subarray sum or exact count, especially with negatives.

Core idea

If $\text{prefix}[j] - \text{prefix}[i] = k$, store earlier prefix values in a map.

```
from collections import defaultdict

def subarray_sum(nums, k):
    count = defaultdict(int)
    count[0] = 1
    prefix = 0
    ans = 0

    for x in nums:
        prefix += x
        ans += count[prefix - k]
        count[prefix] += 1

    return ans
```

- Time: $O(n)$
- Space: $O(n)$
- Memory rule: exact total $\rightarrow$ prefix difference

5. Binary Search

Use when

- Sorted input or monotonic answer space.

Core idea

Search the first valid boundary, not just a value.

```
def first_true(lo, hi, check):
    while lo < hi:
        mid = (lo + hi) // 2
        if check(mid):
            hi = mid
        else:
            lo = mid + 1
    return lo
```

- Time: $O(\log n)$ or $O(\log \text{range} * \text{check_cost})$
- Space: $O(1)$
- Memory rule: monotonic condition $\rightarrow$ binary search

6. Heap

Use when

- Need repeated best item, top k , or streaming rank.

Core idea

A heap keeps the current best candidate cheap to update.

```
import heapq

def top_k_largest(nums, k):
    heap = []
    for x in nums:
        if len(heap) < k:
            heapq.heappush(heap, x)
        elif x > heap[0]:
            heapq.heapreplace(heap, x)
    return sorted(heap, reverse=True)
```

- Time: $O(n \log k)$

- Space: $O(k)$
- Memory rule: repeated best-item access -> heap

7. BFS

Use when

- Unweighted shortest path or level-by-level expansion.

Core idea

BFS explores in layers, so first visit gives shortest step count.

```
from collections import defaultdict, deque

def bfs_distances(n, edges, start):
    graph = defaultdict(list)
    for u, v in edges:
        graph[u].append(v)
        graph[v].append(u)

    dist = {start: 0}
    q = deque([start])

    while q:
        node = q.popleft()
        for nei in graph[node]:
            if nei not in dist:
                dist[nei] = dist[node] + 1
                q.append(nei)

    return dist
```

- Time: $O(V + E)$
- Space: $O(V + E)$
- Memory rule: shortest unweighted path -> BFS

8. Topological Sort

Use when

- Dependency order in a DAG.

Core idea

Nodes with indegree 0 are available now.

```
from collections import defaultdict, deque

def topo_sort(num_nodes, edges):
    graph = defaultdict(list)
    indegree = [0] * num_nodes

    for u, v in edges:
        graph[u].append(v)
        indegree[v] += 1

    q = deque(i for i in range(num_nodes) if indegree[i] == 0)
    order = []

    while q:
        node = q.popleft()
        order.append(node)
        for nei in graph[node]:
            indegree[nei] -= 1
            if indegree[nei] == 0:
                q.append(nei)

    return order if len(order) == num_nodes else []
```

- Time: $O(V + E)$
- Space: $O(V + E)$
- Memory rule: DAG dependency order -> indegree queue

9. Dijkstra

Use when

- Weighted shortest path with non-negative weights.

Core idea

Always expand the currently cheapest reachable state first.

```
import heapq
from collections import defaultdict

def dijkstra(n, edges, start):
    graph = defaultdict(list)
    for u, v, w in edges:
        graph[u].append((v, w))

    dist = [float('inf')] * n
    dist[start] = 0
    heap = [(0, start)]

    while heap:
        curr_dist, node = heapq.heappop(heap)
        if curr_dist > dist[node]:
            continue
        for nei, w in graph[node]:
            new_dist = curr_dist + w
            if new_dist < dist[nei]:
                dist[nei] = new_dist
                heapq.heappush(heap, (new_dist, nei))

    return dist
```

- Time: $O((V + E) \log V)$
- Space: $O(V + E)$
- Memory rule: weighted shortest path -> Dijkstra

10. Dynamic Programming

Use when

- Best answer over many choices with repeated subproblems.

Core idea

Define state, transition, base case. Compute each state once.

```
def coin_change(coins, amount):
    dp = [amount + 1] * (amount + 1)
    dp[0] = 0

    for total in range(1, amount + 1):
        for coin in coins:
            if total - coin >= 0:
                dp[total] = min(dp[total], dp[total - coin] + 1)

    return dp[amount] if dp[amount] != amount + 1 else -1
```

- Time: states times transitions
- Space: number of states
- Memory rule: repeated subproblems -> DP

11. Greedy

Use when

- A local move appears globally safe.

Core idea

Take the best immediate move only if you can justify it never hurts later.

```
def can_jump(nums):
    farthest = 0

    for i, jump in enumerate(nums):
        if i > farthest:
            return False
        farthest = max(farthest, i + jump)

    return True
```

- Time: often $O(n)$
- Space: often $O(1)$
- Memory rule: local best must be provably safe

12. Design Problems

Use when

- Need a class with time-complexity guarantees.

Core idea

Match each operation to the right primitive: hashmap, heap, deque, linked list, or binary search.

```
from bisect import bisect_right
from collections import defaultdict

class TimeMap:
    def __init__(self):
        self.store = defaultdict(list)

    def set(self, key, value, timestamp):
        self.store[key].append((timestamp, value))

    def get(self, key, timestamp):
        arr = self.store[key]
        i = bisect_right(arr, (timestamp, chr(127))) - 1
        return arr[i][1] if i >= 0 else ""
```

- set: $O(1)$ amortized
- get: $O(\log n)$ per key
- Memory rule: design = choose the right internal data structures

Fast Confusion Checks

- Sliding window vs prefix sum:
window for local validity, prefix for exact total
- BFS vs DFS:
BFS for shortest unweighted path, DFS for exploration
- Heap vs quickselect:
heap for repeated top k , quickselect for one k th query
- DP vs greedy:
if local choice is not obviously safe, suspect DP
- Binary search vs two pointers:
monotonic boundary vs moving scan

Last-Minute Quant Interview Reminders

- Start with arrays, strings, heaps, binary search, graphs, and DP.
- Say the invariant before you code.
- For market / time-series flavored questions, think:
rolling window, prefix sums, heap ranking, graph traversal, DP optimization.
- If stuck, ask:
what must be cheap: lookup, rank, path, order, or optimization?